CHAPTER 12

The Evolution of Attribute Methods

By this point in your reading, you've seen plenty of metaprogramming snippets and examples. However, you might still wonder what happens when you use metaprogramming in a real, large system. How do these sophisticated techniques fare in the messy world out there, where code often grows in complexity and evolves in unexpected directions?

To answer this question, we will close our tour with a look at attribute methods, one of Rails' most popular features. Their source code contains a lot of metaprogramming, and it has been changing constantly since the first version of Rails. If we track the history of attribute methods, we'll see what happened as their code became more complicated and nuanced.

One word of warning before we begin: there is plenty of complex code in this chapter, and it would be pointless to explain it in too much detail. Instead, I'll just try to make a point by giving you a high-level idea of what's going on. Don't feel as if you have to understand each and every line of code as you read through the next few pages.

Let's start with a quick example of attribute methods.

Attribute Methods in Action

Assume that you've created a database table that contains tasks.

part2/ar_attribute_methods.rb

```
require 'active_record'
ActiveRecord::Base.establish_connection :adapter => "sqlite3",
                                        :database => "dbfile"

ActiveRecord::Base.connection.create_table :tasks do |t|
  t.string   :description
  t.boolean  :completed
end
```